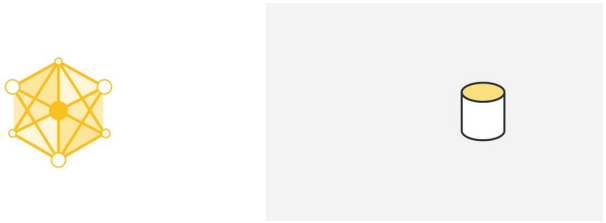
Trabalho 1

Aprendizagem Federada

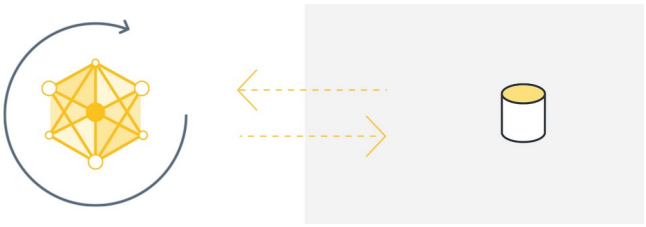
Aprendizado de máquina clássico

Antes de começarmos a discutir a aprendizagem federada, vamos recapitular rapidamente como funciona a maioria dos processos de aprendizagem de máquina atualmente.

Na aprendizagem de máquina, temos um modelo e temos dados. O modelo pode ser uma rede neural (como ilustrado aqui) ou algo diferente, como a regressão linear clássica.

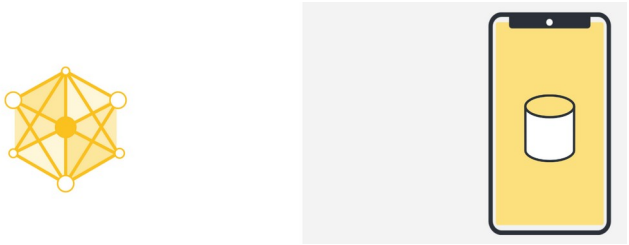


Treinamos o modelo usando os dados para realizar uma tarefa útil. Uma tarefa pode ser detectar objetos em imagens, transcrever uma gravação de áudio ou jogar um jogo como Go.

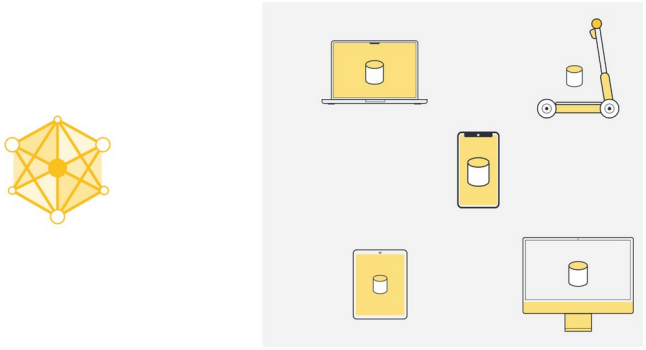


Na prática, os dados de treinamento com os quais trabalhamos não se originam na máquina em que treinamos o modelo.

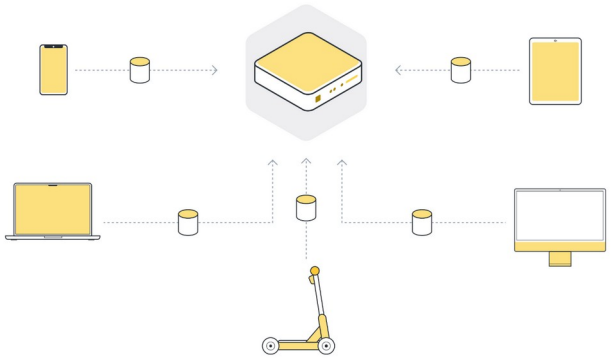
Esses dados são criados "em outro lugar". Por exemplo, os dados podem ter origem em um smartphone, quando o usuário interage com um aplicativo; em um carro, quando ele coleta dados de sensores; em um laptop, quando ele recebe dados do teclado; ou em um alto-falante inteligente, quando alguém tenta cantar uma música.



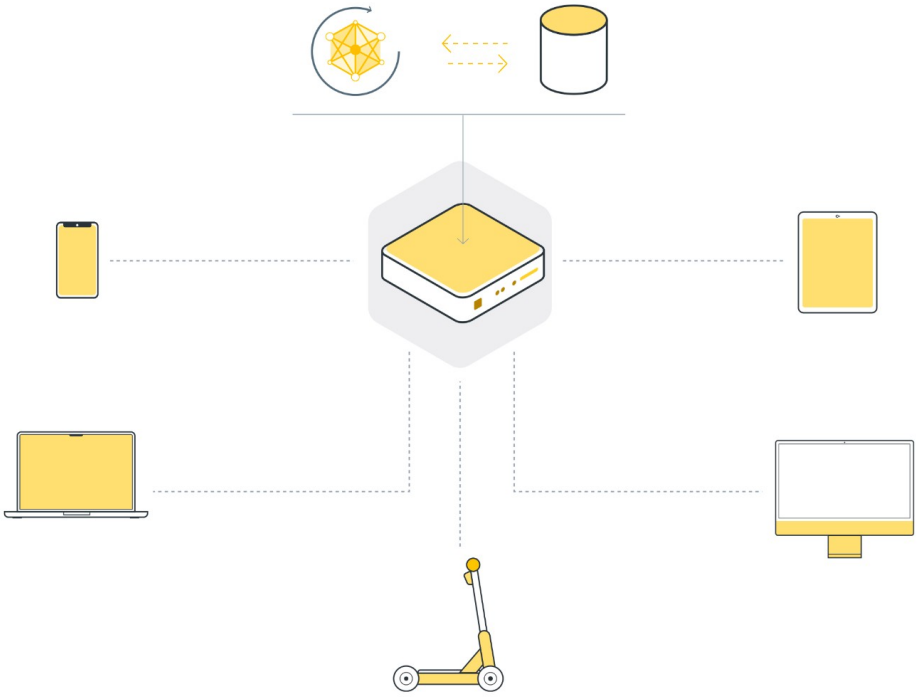
É importante mencionar também que esse "outro lugar" geralmente não se refere a um único local, mas a vários. Podem ser diversos dispositivos executando o mesmo aplicativo. Mas também podem ser várias organizações, todas gerando dados para a mesma tarefa.



Para usar aprendizado de máquina ou qualquer tipo de análise de dados, a abordagem utilizada no passado consistia em coletar todos esses dados em um servidor central. Esse servidor podia estar localizado em um data center ou na nuvem.



Uma vez que todos os dados estejam reunidos em um só lugar, podemos finalmente usar algoritmos de aprendizado de máquina para treinar nosso modelo com esses dados. Essa é a abordagem de aprendizado de máquina na qual sempre nos baseamos.



Desafios da aprendizagem de máquina clássica

Essa abordagem clássica de aprendizado de máquina que acabamos de ver pode ser usada em alguns casos. Ótimos exemplos incluem a categorização de fotos de férias ou a análise de tráfego na web. Casos em que todos os dados estão naturalmente disponíveis em um servidor centralizado.



Mas essa abordagem não pode ser usada em muitos outros casos. Casos em que os dados não estão disponíveis em um servidor centralizado, ou casos em que os dados disponíveis em um servidor não são suficientes para treinar um bom modelo.



Existem muitas razões pelas quais a abordagem clássica de aprendizado de máquina centralizado não funciona para um grande número de casos de uso importantes no mundo real. Essas razões incluem:

- **Regulamentos** : GDPR (Europa), CCPA (Califórnia), PIPEDA (Canadá), LGPD (Brasil), PDPL (Argentina), KVKK (Turquia), POPI (África do Sul), FSS (Rússia), CDPR (China), PDPB (Índia), PIPA (Coreia), APPI (Japão), PDP (Indonésia), PDPA (Singapura), APP (Austrália) e outros regulamentos protegem dados sensíveis contra a transferência. De fato, esses regulamentos às vezes até impedem que organizações individuais combinem os dados de seus próprios usuários para treinamento de aprendizado de máquina, porque esses usuários residem em diferentes partes do mundo e seus dados são regidos por diferentes regulamentações de proteção de dados.
- **Preferência do usuário** : Além da regulamentação, existem casos de uso em que os usuários simplesmente esperam que nenhum dado saia de seus dispositivos, nunca. Se você digita suas senhas e informações de cartão de crédito no teclado digital do seu celular, você não espera que essas senhas acabem no servidor da empresa que desenvolveu esse teclado, certo? Na verdade, esse caso de uso foi o motivo pelo qual o aprendizado federado foi inventado.
- **Volume de dados** : Alguns sensores, como câmeras, produzem um volume de dados tão alto que coletar todos os dados se torna inviável e antieconômico (devido, por exemplo, à largura de banda ou à eficiência da comunicação). Imagine um serviço ferroviário nacional com centenas de estações em todo o país. Se cada uma dessas estações estiver equipada com

diversas câmeras de segurança, o volume de dados brutos gerados pelos dispositivos exigirá uma infraestrutura incrivelmente poderosa e extremamente cara para processamento e armazenamento. E a maior parte desses dados sequer é útil.

Exemplos de situações em que o aprendizado de máquina centralizado não funciona incluem:

- Registros de saúde sensíveis de múltiplos hospitais para treinar modelos de detecção de câncer.
- Informações financeiras de diferentes organizações para detectar fraudes financeiras.
- Dados de localização do seu carro elétrico para uma melhor previsão de autonomia.
- Mensagens criptografadas de ponta a ponta para treinar melhores modelos de preenchimento automático.

A popularidade de sistemas que aprimoram a privacidade, como o navegador Brave ou o mensageiro Signal , demonstra que os usuários se preocupam com a privacidade. De fato, eles escolhem a versão que prioriza a privacidade em detrimento de outras alternativas, caso existam. Mas o que podemos fazer para aplicar aprendizado de máquina e ciência de dados a esses casos, de modo a utilizar dados privados? Afinal, essas são áreas que se beneficiariam significativamente dos recentes avanços em IA.

Aprendizagem Federada

A aprendizagem federada simplesmente inverte essa abordagem. Ela possibilita o aprendizado de máquina em dados distribuídos, movendo o treinamento para os dados, em vez de mover os dados para o treinamento. Aqui está uma explicação resumida:

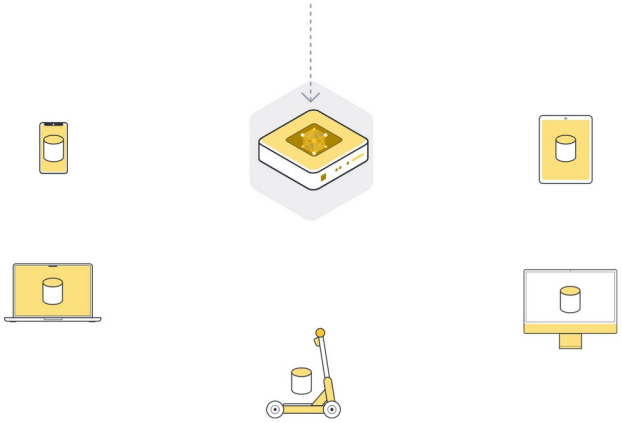
- Aprendizado de máquina centralizado: mova os dados para o processamento.
- Aprendizado federado (de máquina): mover a computação para os dados.

Dessa forma, a Aprendizagem Federada nos permite usar aprendizado de máquina (e outras abordagens de ciência de dados) em áreas onde antes não era possível. Agora podemos treinar excelentes modelos de IA médica, permitindo que diferentes hospitais trabalhem juntos. Podemos solucionar fraudes financeiras treinando modelos de IA com os dados de diferentes instituições financeiras. Podemos criar aplicativos inovadores que aprimoram a privacidade (como mensagens seguras) e que possuem IA integrada superior às suas alternativas que não priorizam a privacidade. E esses são apenas alguns exemplos que nos vêm à mente. À medida que implementamos a Aprendizagem Federada, descobrimos cada vez mais áreas que podem ser reinventadas repentinamente, pois agora têm acesso a vastas quantidades de dados antes inacessíveis.

Então, como funciona exatamente o Aprendizado Federado? Vamos começar com uma explicação intuitiva.

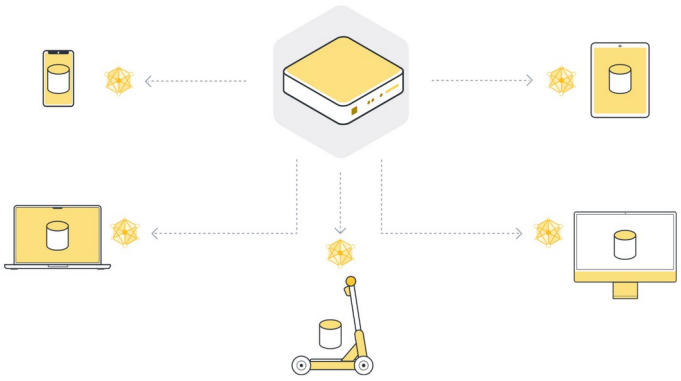
Etapa 0: Inicializar o modelo global

Começamos inicializando o modelo no servidor. Isso é exatamente igual ao aprendizado centralizado clássico: inicializamos os parâmetros do modelo, seja aleatoriamente ou a partir de um ponto de verificação salvo anteriormente.



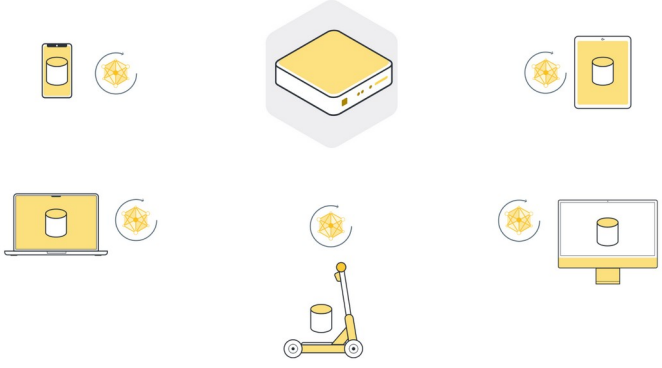
Etapa 1: Enviar modelo para várias organizações/dispositivos conectados (nós clientes)

Em seguida, enviamos os parâmetros do modelo global para os nós clientes conectados (pense em dispositivos de borda, como smartphones, ou servidores pertencentes a organizações). Isso garante que cada nó participante inicie seu treinamento local usando os mesmos parâmetros do modelo. Frequentemente, usamos apenas alguns dos nós conectados em vez de todos. O motivo é que selecionar cada vez mais nós clientes apresenta retornos decrescentes.



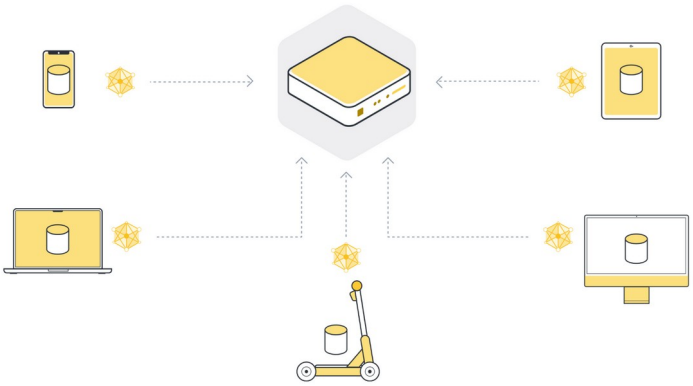
Etapa 2: Treinar o modelo localmente com os dados de cada organização/dispositivo (nó cliente)

Agora que todos os nós clientes (selecionados) possuem a versão mais recente dos parâmetros do modelo global, eles iniciam o treinamento local. Eles usam seu próprio conjunto de dados local para treinar seu próprio modelo local. O treinamento do modelo não prossegue até a convergência completa, mas é realizado por um curto período. Isso pode ser tão curto quanto uma época nos dados locais, ou até mesmo apenas alguns passos (mini-lotes).



Etapa 3: Retornar as atualizações do modelo para o servidor

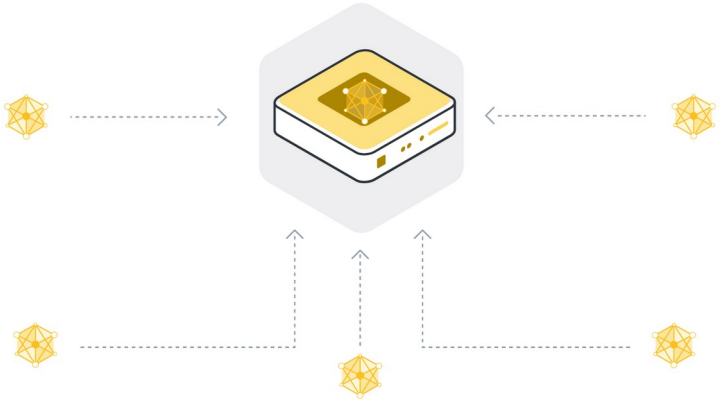
Após o treinamento local, cada nó cliente possui uma versão ligeiramente diferente dos parâmetros do modelo que recebeu inicialmente. Os parâmetros são todos diferentes porque cada nó cliente tem exemplos diferentes em seu conjunto de dados local. Os nós clientes então enviam essas atualizações do modelo de volta para o servidor. As atualizações do modelo que eles enviam podem ser os parâmetros completos do modelo ou apenas os gradientes que foram acumulados durante o treinamento local.



Etapa 4: Agregar as atualizações do modelo em um novo modelo global

O servidor recebe atualizações do modelo dos nós clientes selecionados. Se ele selecionou 100 nós clientes, agora possui 100 versões ligeiramente diferentes do modelo global original, cada uma treinada com os dados locais de um cliente. Mas não seria desejável ter um único modelo que contivesse o aprendizado dos dados de todos os 100 nós clientes?

Para obter um único modelo, precisamos combinar todas as atualizações de modelo recebidas dos nós clientes. Esse processo é chamado de *agregação* e existem muitas maneiras diferentes de realizá-lo. A maneira mais básica é chamada de *Média Federada* (McMahan et al., 2016), frequentemente abreviada como *FedAvg* . *O FedAvg* pega as 100 atualizações de modelo e, como o nome sugere, calcula a média delas. Mais precisamente, ele calcula a *média ponderada* das atualizações de modelo, ponderada pelo número de exemplos que cada cliente usou para treinamento. A ponderação é importante para garantir que cada exemplo de dados tenha a mesma "influência" no modelo global resultante. Se um cliente tiver 10 exemplos e outro tiver 100, então — sem ponderação — cada um dos 10 exemplos influenciaria o modelo global dez vezes mais do que cada um dos 100 exemplos.



Etapa 5: Repita as etapas 1 a 4 até que o modelo convirja

As etapas de 1 a 4 constituem o que chamamos de uma única rodada de aprendizado federado. Os parâmetros do modelo global são enviados aos nós clientes participantes (etapa 1), os nós clientes treinam com seus dados locais (etapa 2), enviam seus modelos atualizados para o servidor (etapa 3) e o servidor, então, agrega as atualizações do modelo para obter uma nova versão do modelo global (etapa 4).

Durante uma única rodada, cada nó cliente que participa dessa iteração treina apenas por um curto período. Isso significa que, após a etapa de agregação (etapa 4), temos um modelo que foi treinado com todos os dados de todos os nós clientes participantes, mas apenas por um curto período. Em seguida, precisamos repetir esse processo de treinamento diversas vezes até chegarmos a um modelo totalmente treinado que tenha um bom desempenho com os dados de todos os nós clientes.

Simulação de aprendizagem federada

Dependências que devem ser instaladas e por que elas são necessárias

Primeiramente baixe o código do trabalha do sigaa. O arquivo deve conter uma pasta codigo.zip, descompacte-o em um diretório codigo que conterá os arquivos fontes. Entre nesse diretório e crie um ambiente python. Para essa criação, recomendo a instalação do gerenciador uv. Clicando no link você será redirecionado para a página de instalação, siga os passos recomendados e deverá instalar corretamente o gerenciador de pacotes e ambientes python. Após a instalação do uv, crie um ambiente virtual dentro do diretório com o comando:

```
uv venv --python 3.14
```

Ative o ambiente virtual com o comando abaixo se for usuário Linux ou Mac:

```
source .venv/bin/activate
```

Caso seja Windows:

```
PowerShell: .\venv\Scripts\Activate.ps1

ou

CMD (Prompt): venv\Scripts\activate.bat
```

Você poderá desativar em momento posterior o ambiente de programação de forma similar executando o comando:

```
deactivate
```

Atualize o pip:

```
pip install --upgrade pip
```

Para executar a simulação, o conjunto mínimo de bibliotecas é flwr-datasets, scikit-learn, numpy e pandas. O flwr-datasets é necessário porque ele fornece o FederatedDataset, usado para baixar e particionar o dataset em clientes simulados; a própria documentação o descreve como a biblioteca para criação e particionamento de datasets em cenários de aprendizado federado. O scikit-learn é

necessário porque o modelo usado na simulação é LogisticRegression, além de métricas como log_loss. O numpy é necessário para manipular vetores e matrizes de parâmetros do modelo durante a agregação. O pandas é necessário porque o código converte as partições para DataFrame com with_format("pandas"), facilitando a seleção das colunas do Iris. Um comando adequado para instalação é:

```
# instala todas as dependências listadas em pyproject.toml

uv sync
```

Ou mais explicitamente:

```
# instala todas as dependências listadas em pyproject.toml

uv pip install -r pyproject.toml
```

Ou ainda mais explícito:

```
# instala todas as dependências de forma explícita

uv pip install flwr-datasets scikit-learn numpy pandas
```

Na prática, flwr-datasets trabalha em conjunto com datasets do ecossistema Hugging Face, e a documentação do Flower Datasets deixa claro que ele baixa e particiona datasets a partir desse ecossistema. Por isso, em alguns ambientes a instalação dessas dependências principais já resolve tudo, mas o ponto mais importante para o leitor é entender o papel de cada pacote: **Flower Datasets cuida dos dados, scikit-learn cuida do modelo, NumPy cuida da matemática matricial e pandas ajuda na manipulação tabular**.

O dataset Iris e o problema tratado

O problema simulado é um problema de classificação multiclasse. O dataset Iris é um conjunto clássico da área de aprendizado de máquina, com 150 amostras, 3 classes e 4 atributos numéricos. As classes representam espécies de flor, e os atributos são medidas das sépalas e pétalas. A documentação oficial do scikit-learn descreve o Iris exatamente como um dataset clássico e simples de classificação multiclasse.

No código, são usadas quatro features: petal_length, petal_width, sepal_length e sepal_width. O rótulo a ser previsto é species. Em outras palavras, o modelo recebe medidas físicas da flor e precisa prever a qual espécie ela pertence. Esse tipo de problema é adequado para uma primeira simulação federada porque o dataset é pequeno, fácil de interpretar e suficientemente estruturado para demonstrar como os clientes treinam localmente e como o servidor agrega os parâmetros. O quickstart oficial do Flower com scikit-learn usa exatamente o Iris com esse tipo de configuração. O uso do Iris também é didaticamente vantajoso porque permite focar no mecanismo do aprendizado federado sem a complexidade de imagens grandes, redes neurais profundas ou pré-processamentos pesados. Aqui, o interesse principal não é “resolver um problema difícil”, mas mostrar claramente a sequência: particionar dados, treinar localmente, agregar parâmetros e atualizar o modelo global.

Explicação das funções geradas no código

O código descrito abaixo é o **fl_sim_manual.py**.

A função carregar_clientes é responsável por preparar os dados para a simulação. Ela cria um IidPartitioner, instancia o FederatedDataset com o Iris e divide o split "train" em várias partições, uma para cada cliente. Ou seja, ele pega o dataset de treino original e divide os dados entre os clientes. A documentação do IidPartitioner informa que ele cria partições amostradas aleatoriamente do dataset, e seguindo a mesma distribuição de dados entre os clientes, o que corresponde ao cenário IID usado no código. Depois disso, a função separa cada partição em treino e teste locais, normalmente em 80% e 20%. Assim, cada cliente terá 80% dos seus dados usados para treino local e 20% utilizado para testar o modelo global agregado.

A função criar_modelo_inicial cria o modelo de regressão logística que será compartilhado entre os clientes. O uso de max_iter=1 e warm_start=True serve para que cada rodada local funcione como um pequeno passo de treinamento sobre os parâmetros já existentes, em vez de reinicializar o modelo do zero a cada vez. A documentação do scikit-learn descreve LogisticRegression como um classificador de regressão logística regularizada, apropriado para esse tipo de tarefa supervisionada.

As funções obter_parametros e definir_parametros existem para separar claramente o estado do modelo da lógica de treino. obter_parametros extrai os pesos do modelo; definir_parametros coloca esses valores de volta em outro modelo. Em aprendizado federado, isso é essencial porque o servidor não envia o objeto do modelo inteiro para os clientes; ele envia apenas os parâmetros globais que representam o estado atual do modelo. Essa separação deixa explícito o que está sendo trocado entre servidor e clientes.

A função treino_local representa o comportamento de um cliente. Ela recebe os parâmetros globais atuais, cria uma cópia local do modelo, aplica esses parâmetros e executa o treinamento usando apenas os dados daquele cliente. No final, ela devolve os novos parâmetros e métricas locais, como perda e acurácia. Conceitualmente, esta função é o “lado cliente” do aprendizado federado: ela mostra que o dado continua local, e que apenas o resultado do treinamento é compartilhado.

A função agregar_fedavg representa o papel do servidor na etapa de agregação. Ela recebe as atualizações de todos os clientes e calcula uma média ponderada pelo número de exemplos de treino de cada cliente. Essa é a ideia central do FedAvg.

A função avaliar_modelo_global junta os conjuntos de teste locais dos clientes e avalia o modelo global agregado. Ela é importante porque permite observar se o modelo global está melhorando de rodada para rodada. Já a função simular_aprendizado_federado coordena todo o processo: carrega os clientes, cria o modelo inicial, executa as rodadas, chama o treino local de cada cliente, agrega os parâmetros e mostra as métricas ao final de cada rodada. Ela é o “controlador” da simulação.

Explicação da simulação com foco nos passos do aprendizado federado

A simulação tenta reproduzir os passos centrais do aprendizado federado, mas de forma manual e didática. O primeiro passo é a distribuição dos dados entre clientes. Em vez de um único dataset centralizado usado por um único processo de treino, o Iris é particionado com FederatedDataset e IidPartitioner, de modo que cada cliente receba apenas sua parte. Essa etapa representa a ideia de que, em um sistema federado, os dados estão distribuídos entre dispositivos ou organizações.

O segundo passo é a inicialização do modelo global no servidor. O servidor começa com um conjunto inicial de parâmetros, que será enviado a todos os clientes na primeira rodada. Esse modelo global é a referência comum da federação: todos os clientes começam cada rodada a partir

do mesmo ponto. No código, isso aparece quando o servidor mantém parametros_globais e os repassa para cada cliente antes do treino local.

O terceiro passo é o treinamento local em cada cliente. Cada cliente usa apenas sua partição de treino e ajusta localmente o modelo recebido do servidor. É aqui que o aprendizado realmente acontece sobre os dados. O aspecto federado aparece exatamente no fato de que os dados não saem do cliente; o que volta ao servidor são apenas os parâmetros atualizados e algumas métricas. Essa separação entre “dados locais” e “parâmetros compartilhados” é uma das ideias centrais do paradigma federado.

O quarto passo é a agregação no servidor. Depois que todos os clientes terminam seu treino local, o servidor coleta os parâmetros devolvidos e calcula uma média ponderada. O peso de cada cliente depende do número de exemplos usados no treino local, o que é coerente com a lógica do FedAvg: clientes com mais dados contribuem proporcionalmente mais para o novo modelo global. Esse é o momento em que o conhecimento aprendido localmente é consolidado em um único modelo global.

O quinto passo é a repetição por rodadas. O modelo global recém-agregado é enviado novamente aos clientes na rodada seguinte, que repetem o processo de treino local. Ao longo das rodadas, o modelo global tende a melhorar, desde que a agregação e o treinamento local estejam coerentes. No código, isso aparece no laço principal da simulação, em que o servidor distribui parâmetros, recebe atualizações, agrega resultados e avalia o modelo global após cada rodada. Esse ciclo é justamente o coração do aprendizado federado.

É importante notar que esta implementação é uma simulação simplificada. Não há comunicação em rede, paralelismo real, falhas de clientes, privacidade diferencial ou criptografia. Mesmo assim, ela cumpre muito bem o papel pedagógico de mostrar as etapas fundamentais: cliente treina localmente, servidor agrega, e o processo se repete. Para um iniciante, isso costuma ser mais útil do que começar diretamente por uma infraestrutura mais complexa.

Simulação de aprendizagem federada com gRPC

Instruções para o aluno

Neste trabalho, parte do código foi disponibilizada de forma propositalmente incompleta. Seu objetivo é **complementar as funções indicadas com TODO**, seguindo a lógica do sistema distribuído baseado em cliente, servidor, gRPC e aprendizado federado.

Ao longo da atividade, você deverá implementar as funções passo a passo, respeitando a estrutura já fornecida no código. A ideia não é reescrever tudo do zero, mas entender o papel de cada função e completar apenas o que está faltando.

Durante a implementação, é fundamental que você **execute os testes unitários** disponíveis no projeto. Eles foram preparados para servir como guia de construção: cada teste verifica uma parte específica do comportamento esperado e ajuda a identificar se sua implementação está correta antes de avançar para a próxima etapa.

Recomenda-se fortemente que você:

- leia atentamente os comentários e dicas deixados no código;
- implemente uma função por vez;
- execute os testes relacionados após cada etapa;
- só avance quando a etapa atual estiver funcionando corretamente.

Os testes não servem apenas para “dar nota”, mas para orientar sua construção. Se algum teste falhar, leia com atenção a mensagem retornada e revise a lógica da função correspondente.

Em resumo, sua tarefa é:

- completar as funções indicadas;
- seguir a ordem sugerida no material;
- usar os testes unitários para validar cada etapa;
- e manter sua implementação compatível com a arquitetura fornecida.

Antes de entregar, certifique-se de que:

- todas as funções obrigatórias foram implementadas;
- os testes unitários passam corretamente;
- e o código segue as instruções e a estrutura proposta no trabalho.